Professional Training in Cybersecurity I

Module 0 — Session 5

Python Essentials for Cybersecurity

Mohamed Gebril

Session Objective

By the end you can do each of these — on your own machine, not just on the slides:

Run — Python, two ways

in the REPL, and as a script the shell can execute

Isolate — each project's packages

in a virtual environment, as Kali's own message advises

Write — decisions and loops

types, text, conditions, loops, lists and dictionaries

Structure — code into functions

modules, files, and errors handled without a crash

Handle — raw bytes

bytes versus text, hexadecimal, and encoding

Build — a command-line tool

arguments with argparse, addresses with ipaddress

Session Agenda

Part 1 — The Python Environment
what Python is and why this course uses it, the interpreter, scripts, and virtual environments

Part 2 — Values, Types, and Text
the core types, names, input and output, numbers, and strings

Part 3 — Control Flow and Collections
conditions and loops, lists, tuples, dictionaries and sets

Part 4 — Functions and Modules
parameters, return values, scope, and import

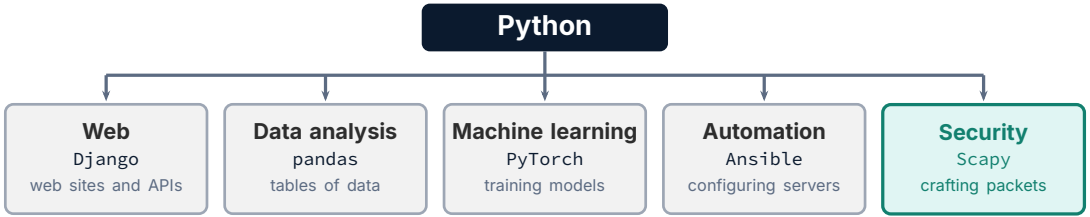
Part 5 — Files and Exceptions
reading and writing files, and handling errors

Part 6 — Bytes and Binary Data
bytes versus text, hexadecimal, and encoding

Part 7 — Command-Line Tools
arguments with argparse, and addresses with ipaddress

The Python Environment

Python: A General-Purpose Language and Its Fields



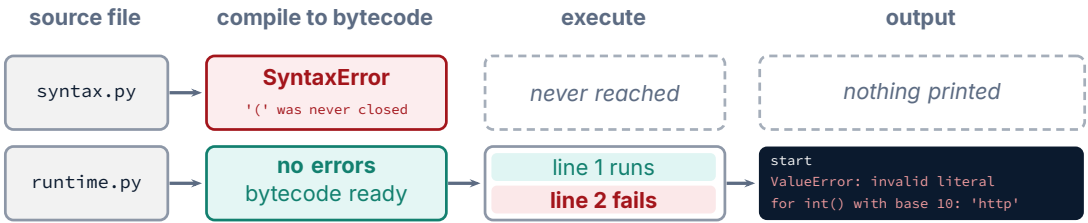
- **Python is a high-level, general-purpose programming language**: one language for many kinds of problem, run by an interpreter rather than compiled into an executable first.
- Guido van Rossum released it in 1991. In the 2025 Stack Overflow Developer Survey, 57.9% of respondents used it, fourth after JavaScript, HTML/CSS and SQL, and it came **first among people learning to code**, at 71.8%.
- **This course is Python 3 only.** Python 2 reached its end of life on January 1, 2020, yet much published security code still uses it, so an old example can fail with errors that look like your own mistake.

Why This Course Uses Python

	Python	C	bash
Writing a tool	a few lines, in minutes	memory managed by hand	quick until the logic grows
Raw bytes	a built-in bytes type	native: bytes are memory	awkward: built for text
Running speed	slower: interpreted	fastest: machine code	slow at computation
In this course	**every tool you build**	**where a topic needs it**	**running the tools**

- **A language is a tool chosen for a job, and this course chooses Python for how fast a tool can be written and changed**, not for how fast it runs.
- Where running speed or direct access to memory is the point, the course moves to C. Bash stays at what it does best, which is starting other programs.
- Scapy and sqlmap, two tools installed on Kali, are Python programs, so once you read Python you can read how they work.

The Interpreter: Source, Bytecode, Execution



- **An interpreter is a program that runs source code directly**, with no build step of yours in between. `python3` first compiles the whole file to **bytecode**, the internal form it runs, then executes it from the top.
- **A syntax error stops the file at the compile stage, so no line runs at all.** A runtime error stops it partway, after every earlier line has already done its work.
- The last line of the error names its kind, and the `File …, line N` line above it says where to look.

The Interactive Interpreter and the Script

kali@kali:~/python-essentials$

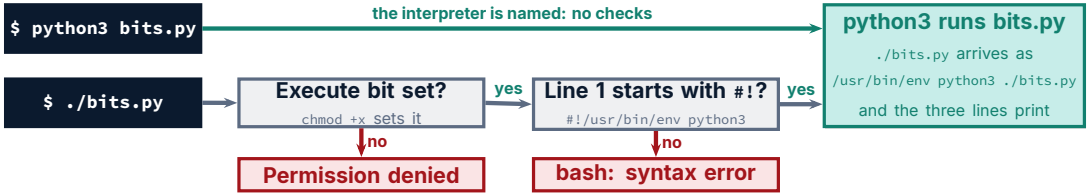
```
$ python3
   [2 lines: the version banner]
>>> 4 * 8
32              # the value, echoed
>>> print("A byte has", 8, "bits")
A byte has 8 bits
>>> exit()
```

kali@kali:~/python-essentials$

```
$ cat bits.py
print("An IPv4 address has", 4, "bytes")
print("A byte has", 8, "bits")
print("So an address has", 4 * 8, "bits")
$ python3 bits.py   # top to bottom
An IPv4 address has 4 bytes
A byte has 8 bits
So an address has 32 bits
```

- **The interactive interpreter, or REPL, reads a line, evaluates it and prints its value**, then waits for the next one. The name is those steps: read, evaluate, print, loop.
- **A script is a file of Python lines that `python3` runs from top to bottom, then exits.** Each `print` in `bits.py` writes one line, in the order the lines are written.
- Try a line in the REPL, and keep anything worth running twice in a `.py` file. A script shows only what it prints, and `exit()` leaves the REPL.

Running a Script: The Shebang and the Execute Bit




```
kali@kali:~/python-essentials$
$ ./bits.py
bash: ./bits.py: Permission denied     # gate 1 fails
$ chmod +x bits.py                     # set the execute bit
$ ./bits.py                            # no shebang yet
./bits.py: line 1: syntax error near unexpected token `"An IPv4 address has",'
./bits.py: line 1: `print("An IPv4 address has", 4, "bytes")'  # bash ran a Python line
```

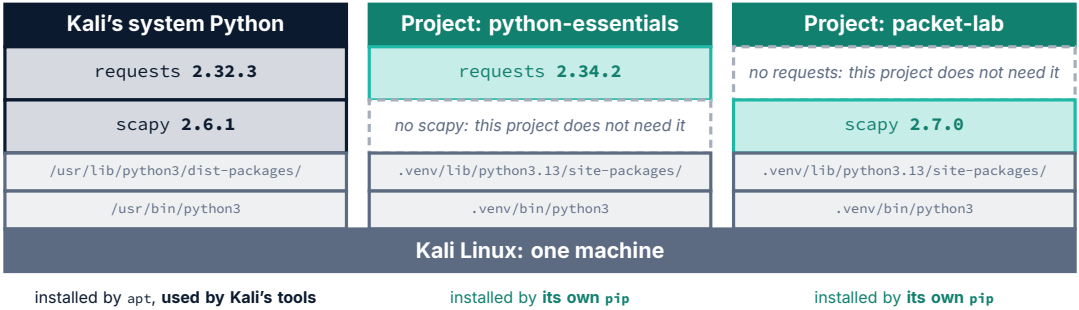
- **The execute bit is a permission that marks a file as a program.** A new file does not have it, so bash reports `Permission denied` until `chmod +x bits.py` sets it.
- **The shebang is a first line starting with `#!` that names the program to run the file with.** Without it, bash runs the file as a bash script and fails on the first Python line.

Packages and `pip`: Installing Code Someone Else Wrote



- **A package is code someone else wrote and published, so a program can import it instead of writing it again.** `requests`, for example, fetches web pages over HTTP.
- **`pip` is Python's package installer.** It downloads a package from PyPI, the Python Package Index, together with every package that one needs, and writes them all into a `site-packages` folder.
- **Every Python on a machine has its own package folder, and `pip` writes into the folder of the Python it belongs to.** A package installed for one Python is not seen by another.

Virtual Environments: One Private Python per Project



- **A virtual environment is a private Python for one project, with its own `python3`, its own `pip` and its own package folder.** It lives inside the project, in a folder usually named `.venv`.
- **Each project installs the versions it needs, and nothing reaches Kali's own Python or another project.** This one machine holds `requests` at two versions and Scapy at two versions, at the same time.
- A new environment holds only `pip`. When one breaks, delete its folder and create it again.

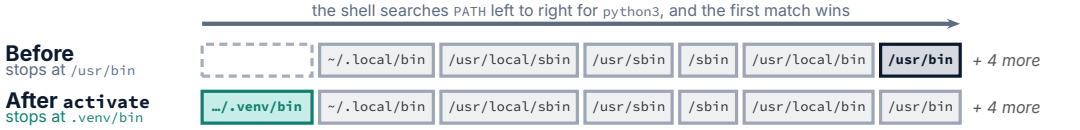
Why Kali's System Python Refuses `pip install`




```
kali@kali:~/python-essentials$
$ pip install requests
error: externally-managed-environment
  [19 lines: the three ways out, apt, a virtual environment, or pipx]
note: If you believe this is a mistake, please contact your Python installation
or OS distribution provider. You can override this, at the risk of breaking your
 Python installation or OS, by passing --break-system-packages.
hint: See PEP 668 for the detailed specification.
```

- **Kali's own Python belongs to `apt`, the system package manager, and Kali's tools depend on the versions `apt` installed.** This command would put `requests` 2.34.2 ahead of `apt`'s 2.32.3, and those tools would load the new one.
- So Kali marks its Python as externally managed, and `pip` refuses, citing PEP 668. The message names three ways out: `apt`, a virtual environment, or `pipx`. This course uses a virtual environment.
- **`--break-system-packages` does exactly what its name says**, so it is the one answer to ignore.

Creating and Activating a Virtual Environment



kali@kali:~/python-essentials$

```
$ python3 -m venv .venv
$ which python3
/usr/bin/python3
$ source .venv/bin/activate
(.venv) $ which python3
/home/kali/python-essentials/.venv/bin/python3
```

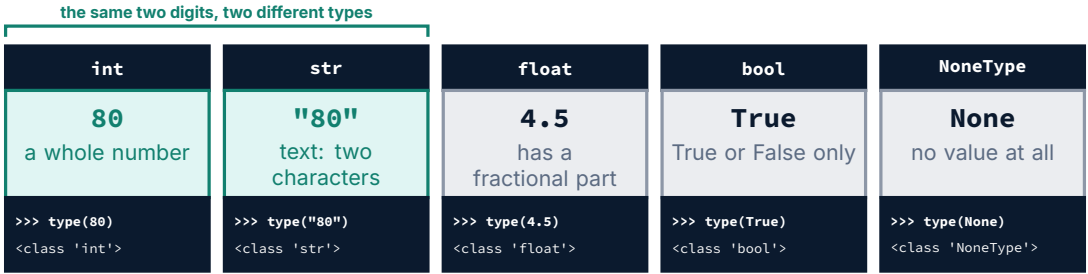
kali@kali:~/python-essentials$

```
(.venv) $ pip install requests
  [16 lines: pip collects requests and the four it needs]
Successfully installed certifi-2026.7.22
charset_normalizer-3.5.2 idna-3.20
requests-2.34.2 urllib3-2.8.0 # in .venv
```

- **`python3 -m venv .venv` creates the environment, and `source .venv/bin/activate` switches it on in this terminal.** The prompt then starts with `(.venv)`, and `deactivate` switches it off.
- **`PATH` is the list of folders the shell searches, left to right, for a command typed by name.** Activation puts `.venv/bin` first, so `python3` and `pip` now mean the environment's own.
- `pip install` now writes into `.venv` and nowhere else. `env` searches `PATH` too, which is why a shebang reads `#!/usr/bin/env python3`: the script runs in the environment you switched on.

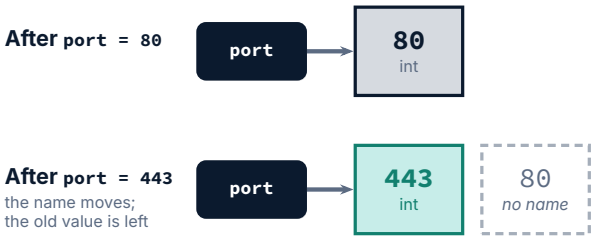
Values, Types, and Text

Values and Types: Every Value Has a Type



- **A value is one piece of data a program works with, and its type says what kind of data it is and what can be done with it.** `type()` asks Python for a value's type.
- **`80` and `"80"` are different values.** The first is an `int`, a number you can add to. The quotes make the second a `str`, two characters of text. A port typed at the keyboard or read from a file arrives as text.
- A `float` is a number with a fractional part, a `bool` is `True` or `False`, and `None` is the one value that means no value at all.

Variables: Names Bound to Values




```
kali@kali:~/python-essentials$
>>> port = 80        # bind the name
>>> port
80                   # its value
>>> port = 443       # move the name
>>> port
443
```

- **A variable is a name bound to a value.** `port = 80` makes the name `port` refer to the value `80`, and typing the name gives that value back.
- **`=` is assignment: it binds the name on its left to the value on its right.** Assigning again moves the name to the new value, and the old value is left with no name.
- A name is made of letters, digits and underscores and cannot start with a digit. Case counts, so `port` and `Port` are two different names. Write them in lowercase, as in `next_port`.

Output, Input, and Comments



~/python-essentials/ask.py

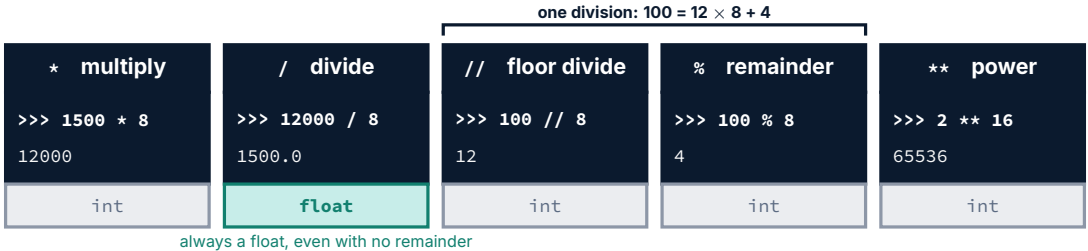
```
1 # ask.py: ask for a port and print it back  # skipped
2 port = input("Port: ")                      # always a str
3 print("You typed", port)
4 print(type(port))
5 print("10.0.0.7", port, sep=":")            # not a space
6 print("Checking", end=" ")                  # no new line
7 print("done")
```

kali@kali:~/python-essentials$

```
$ python3 ask.py
Port: 443          # typed
You typed 443
<class 'str'>
10.0.0.7:443
Checking done
```

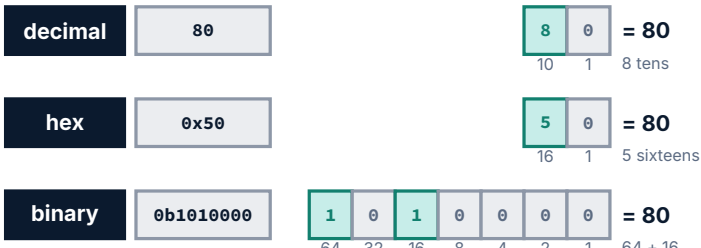
- **`print` writes its values on one line with a space between them, then ends the line.** `sep=":"` replaces the space, and `end=" "` replaces the line ending, so the next `print` carries on the same line.
- **`input` shows a prompt, waits for a line and Enter, and returns the line as a `str`, without the Enter.** Typing `443` gives the text `'443'`, not the number.
- **A `#` starts a comment, and Python ignores everything from it to the end of the line.** A comment is for the person reading the code, like the first line of `ask.py`.

Numbers and Arithmetic



- **Python has two kinds of number: `int` for whole numbers and `float` for numbers with a fractional part.** An `int` has no size limit, and `2 ** 16` counts the port numbers from 0 to 65535.
- **`/` always gives a `float`, even when the division comes out even, so whole-number work uses `//` and `%`.** `100 // 8` and `100 % 8` say 100 bytes fill 12 blocks of 8, with 4 left over.
- A `float` is stored in binary, so most fractions are only close: `0.1 + 0.2` gives `0.30000000000000004`. Count bytes, ports and addresses with `int`.

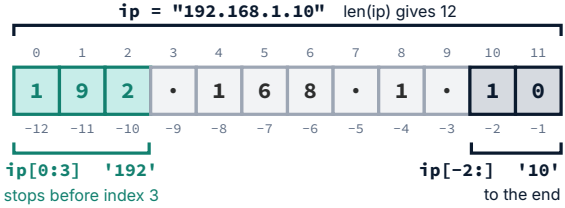
Integer Literals: Decimal, Hex, Binary




```
kali@kali:~/python-essentials$
>>> 0x50
80
>>> 0b1010000
80
>>> 0x50 == 80
True
>>> hex(80)
'0x50'          # a str
>>> bin(80)
'0b1010000'
```

- **A literal is a value written directly in the code, and an integer literal can be written in three bases**: decimal by default, hexadecimal after `0x`, binary after `0b`. All three make the same `int`, so `0x50 == 80` is `True`.
- Each digit is worth the base raised to its position from the right: `0x50` is 5 sixteens and no ones, and `0b1010000` is 64 + 16.
- **`hex()` and `bin()` go the other way, from an `int` to its spelling, and return a `str` with the prefix.** Network tools show bytes in hexadecimal, so you will read `0x` values often.

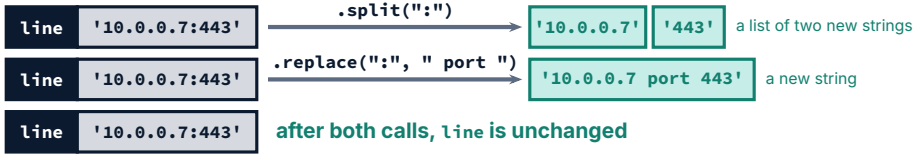
Strings: Text as a Sequence of Characters




```
kali@kali:~/python-essentials$
>>> print("host\t10.0.0.7\nport\t443")
host    10.0.0.7
port    443       # a tab, a new line
>>> banner = """scanner v1
... usage: scan.py HOST"""
>>> print(banner)
scanner v1
usage: scan.py HOST   # two lines
```

- **A string, type `str`, is text: a sequence of characters between quotes.** Single and double quotes make the same string, and `len()` counts the characters.
- **An index picks one character, counting from 0 at the left or from -1 at the right. A slice `[start:stop]` picks a run that stops just before `stop`**, and leaving out `stop` runs to the end.
- Inside quotes, `\n` is a new line and `\t` a tab. Three quotes let a string run over several lines, and the REPL shows `...` while the string is still open.

String Methods and Immutability



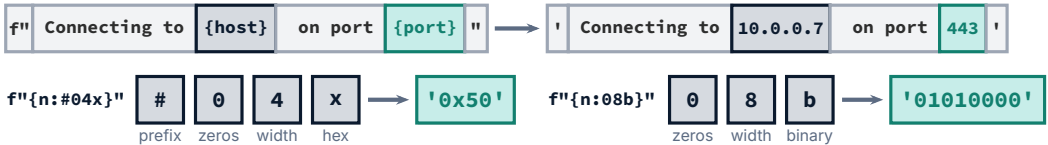
kali@kali:~/python-essentials$

```
>>> line = "10.0.0.7:443"
>>> line.startswith("10.")
True   # starts with 10.
>>> ":" in line
True   # a colon is inside
```

```
>>> "  GET /index.html  ".strip()
'GET /index.html'   # both ends trimmed
>>> "SSH".lower()
'ssh'               # lowercase
```

- **A method is an operation that belongs to a value, called with a dot after it: `line.split(":")`.** Every `str` has methods for the jobs text needs, and `in` tests whether one string occurs inside another.
- **A string never changes once it is made.** `replace` returns a new string and leaves `line` as it was, so to keep the result, assign it: `line = line.replace(":", " port ")`.
- Methods are how Python's built-in values are used. Later tools call methods on files, bytes and addresses in exactly the same way.

Formatting With f-Strings



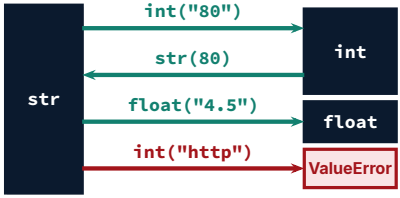
kali@kali:~/python-essentials$

```
>>> host = "10.0.0.7"
>>> port = 443
>>> f"Connecting to {host} on port {port}"
'Connecting to 10.0.0.7 on port 443'    # a str
>>> print(f"Connecting to {host} on port {port}")
Connecting to 10.0.0.7 on port 443      # printed
```

```
>>> n = 80
>>> f"{n:#04x}"
'0x50'
>>> f"{n:08b}"
'01010000'
```

- **An f-string is a string with an `f` before its opening quote, and each `{}` inside it is replaced by the value of what it holds.** The text around the braces stays as written, so one line builds a whole message.
- **A colon inside the braces starts a format spec, which sets how the value is written**: `x` for hexadecimal, `b` for binary, a width, `0` to pad with zeros and `#` for the prefix. The width counts the prefix, so `#04x` gives `0x50`.
- **An f-string turns each value into text as it inserts it, so text and numbers mix with no conversion written.** Joining a `str` and an `int` with `+` fails with a `TypeError` instead.

Converting Between Types




```
kali@kali:~/python-essentials$
>>> int("80")
80
>>> str(80)
'80'
>>> float("4.5")
4.5
>>> int("http")
Traceback (most recent call last):
  [3 lines: where it happened, and the line itself]
ValueError: invalid literal for int() with base 10: 'http'
```

- **A conversion builds a new value of another type from an existing one, and each type's name is the function that converts to it**: `int("80")` gives the number 80, `str(80)` the text `'80'`, and `float("4.5")` the number 4.5.
- **Python never converts a value on its own, so text stays text until you convert it.** A port read with `input()` is a `str`, and needs `int()` before any arithmetic.
- **A conversion fails when the text is not a number of that kind**: `int("http")` raises a `ValueError`. Catching it, so that bad input does not stop the program, comes later in the session.

Booleans and Comparison



with `port = 70000`	
`0 <= port <= 65535`	`False`
`port == 80 or port == 443`	`False`
`port != 80`	`True`
`not port == 80`	`True`

`bool()` of a value		
`bool(0)`	`False`	zero
`bool("")`	`False`	the empty string
`bool(None)`	`False`	None
`bool("0")`	`True`	one character: not empty

- **A `bool` has two values, `True` and `False`, and a comparison produces one**: `==`, `!=`, `<`, `<=`, `>`, `>=`. Comparisons chain, so `0 <= port <= 65535` tests the whole range at once.
- **`=` binds a name and `==` compares two values.** `and`, `or` and `not` combine tests: `port == 80 or port == 443` is `True` for either web port.
- **Where Python expects a `bool`, every value counts as true except zero, the empty string, `None` and `False`.** The string `"0"` has one character, so it counts as true.

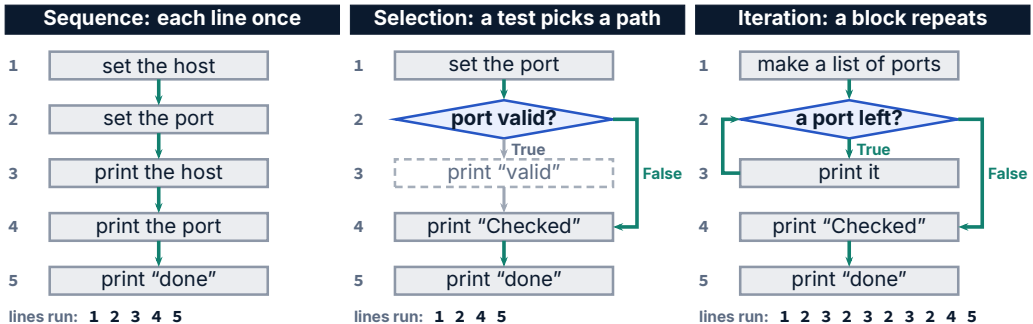
Reading a Traceback

```
kali@kali:~/python-essentials$
$ python3 add.py
Port: 443
Traceback (most recent call last):
  File "/home/kali/python-essentials/add.py", line 2, in <module>   2 where: file and line
    next_port = port + 1                                            3 the line that failed
                ~~~~~^~~
TypeError: can only concatenate str (not "int") to str              1 the error: read it first
$ python3 add_fixed.py                                                # int(port) + 1
Port: 443
Next port: 444
```

- **A traceback is Python's report of the error that stopped a program, and it is read from the bottom up.** Its last line names the kind of error and says what went wrong.
- **The `File` line above it gives the file and the line number, and under it Python prints that line, with carets under the part that failed**: here, `port + 1`.
- The fix follows from the message: `input` returned a `str`, so convert it before adding. `add_fixed.py` uses `int(port) + 1` and prints `444`.

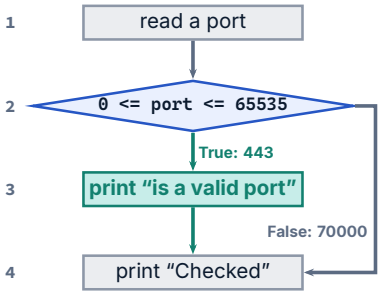
Control Flow and Collections

Control Flow: The Order in Which Lines Run



- **Control flow is the order in which a program's lines run.** Every script so far was a **sequence**: each line ran once, from the top down, as in the left chart.
- **Selection lets a test choose which lines run, and iteration repeats a block.** In the middle, the test is `False`, so line 3 never runs. On the right, lines 2 and 3 run once per port, then line 2 runs once more and finds none.
- **A collection holds many values under one name**, like the list of ports the loop works through. The statements that change the order and the collections they work on are the two halves of this topic.

Conditions: `if` and the Indented Block



~/python-essentials/check.py

```
1 port = int(input("Port: "))
2 if 0 <= port <= 65535:                # the test
3     print(port, "is a valid port")    # the block
4 print("Checked", port)
```

kali@kali:~/python-essentials$

```
$ python3 check.py
Port: 443
443 is a valid port  # True: block ran
Checked 443
$ python3 check.py
Port: 70000
Checked 70000        # False: block skipped
```

- **An `if` statement runs a block only when its test is `True`.** The test comes after `if`, a colon ends the line, and Python checks the test once, when the run reaches it.
- **A block is a group of lines indented under the line that owns it**, four spaces deep by convention. The block ends at the first line back at the outer indentation, so `Checked` prints on every run.
- **Any expression that gives `True` or `False` can be the test**: a comparison, an `in`, or several joined with `and`, `or` and `not`.

Choosing One Path: `elif` and `else`



~/python-essentials/classify.py

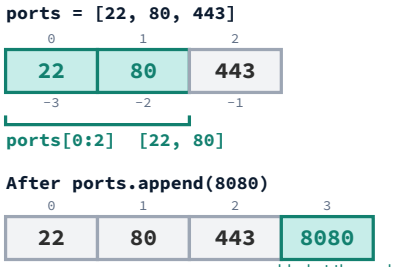
```
1 port = int(input("Port: "))
2 if port < 0:
3     print(port, "is negative")
4 elif port <= 65535:  # only if port < 0 was False
5     print(port, "is a valid port")
6 else:
7     print(port, "is too large")
```

kali@kali:~/python-essentials$

```
$ python3 classify.py
Port: -1
-1 is negative              # if
$ python3 classify.py
Port: 443
443 is a valid port         # elif
$ python3 classify.py
Port: 70000
70000 is too large          # else
```

- **`elif` adds a test that is tried only when every test above it was `False`, and `else` takes whatever is left.** Python tries the tests from the top and runs the first block whose test is `True`. The rest are skipped, and their tests are not even evaluated.
- **The order of the tests is part of the logic.** `port <= 65535` is also `True` for `-1`, but `-1` never reaches it, because `port < 0` caught it first. Each run prints exactly one line.

Lists: An Ordered, Changeable Sequence




```
kali@kali:~/python-essentials$
>>> ports = [22, 80, 443]
>>> ports[0]
22
>>> ports[-1]
443
>>> ports[0:2]
[22, 80]
>>> len(ports)
3
>>> 443 in ports
True
>>> ports.append(8080)                    # nothing printed
>>> ports
[22, 80, 443, 8080]                       # changed in place
```

- **A list, type `list`, is an ordered sequence of values between square brackets, separated by commas.** Indexes, slices, `len()` and `in` work on it exactly as they do on a string.
- **A list can change in place, and a string cannot.** `append` adds one value to the end of the same list and returns nothing, so the REPL prints nothing. A string method returns a new string instead.

The `for` Loop: One Block, Once per Item

pass	`port` holds	line 3 prints
1	22	Checking 22
2	80	Checking 80
3	443	Checking 443
after	no item left: the loop ends	Checked 3 ports done

~/python-essentials/repeat.py

```
1 ports = [22, 80, 443]
2 for port in ports:           # once per item
3     print("Checking", port)  # the block
4 print("Checked", len(ports), "ports")
5 print("done")
```

kali@kali:~/python-essentials$

```
$ python3 repeat.py
Checking 22
Checking 80
Checking 443
Checked 3 ports            # after the loop: once
done
```

- **A `for` loop runs its block once for each item of a sequence, binding a name to the item before each pass.** Each pass is one **iteration**, and any sequence works: a list, or a string, one character at a time.
- **A trace table follows a run one iteration per row**: what `port` holds, and what line 3 prints with it. Writing one by hand is how you check what a loop will do before you run it.
- **When the items run out, the loop ends** and the run carries on at the first line after the block, which prints `Checked 3 ports` once.

Counting With `range`

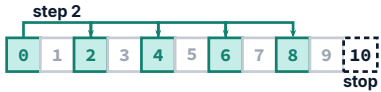
```
>>> list(range(5))
[0, 1, 2, 3, 4]
```




```
>>> list(range(1, 5))
[1, 2, 3, 4]
```




```
>>> list(range(0, 10, 2))
[0, 2, 4, 6, 8]
```



~/python-essentials/hosts.py

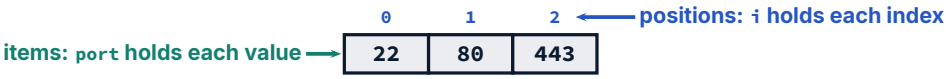
```
1 for i in range(1, 5):  # 1 to 4
2     print(f"10.0.0.{i}")
```

kali@kali:~/python-essentials$

```
$ python3 hosts.py
10.0.0.1
10.0.0.2
10.0.0.3
10.0.0.4
```

- **`range` produces whole numbers from a start up to, but never including, a stop**, moving by a step. `range(5)` starts at 0, `range(1, 5)` at 1, and `range(0, 10, 2)` counts in twos. `list()` shows them.
- **A `for` loop over a `range` repeats a block a set number of times**, with the name holding the count. `hosts.py` builds four addresses from `i` with an f-string.

One `for`, Two Uses: Items and Positions



~/python-essentials/items.py

```
1 ports = [22, 80, 443]
2 for port in ports:      # each item
3     print("port", port)
```

kali@kali:~/python-essentials$

```
$ python3 items.py
port 22
port 80
port 443
```

~/python-essentials/positions.py

```
1 ports = [22, 80, 443]
2 for i in range(len(ports)):    # 0, 1, 2
3     print("index", i, "holds", ports[i])
```

kali@kali:~/python-essentials$

```
$ python3 positions.py
index 0 holds 22
index 1 holds 80
index 2 holds 443
```

- **Python has one `for` loop, and it always walks a sequence one item at a time.** Over `ports` it hands each pass a port. Over `range(len(ports))` it hands each pass an index, 0 to 2.
- **Loop over the items when you need only the values, and over the positions when you also need the index.** `ports[i]` reaches the item at index `i`, as `positions.py` prints it.

Loops That Stop or Skip: `while`, `break`, `continue`

~/python-essentials/retry.py

```
1  while True:
2      port = int(input("Port: "))
3      if 0 <= port <= 65535:
4          break
5      print(port, "is out of range, try again")
6  print("Using port", port)
```

back to the test

break: leave the loop

kali@kali:~/python-essentials$

```
$ python3 retry.py
Port: 70000
70000 is out of range, try again
Port: 443
Using port 443
```

~/python-essentials/filter.py

```
1  for port in [22, 70000, 443]:
2      if port > 65535:
3          continue
4      print(port, "is a valid port")
```

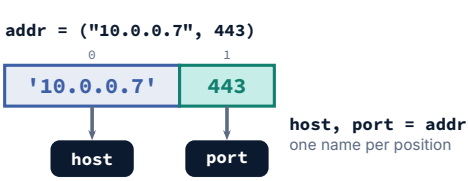
continue: next pass

kali@kali:~/python-essentials$

```
$ python3 filter.py
22 is a valid port
443 is a valid port     # 70000 skipped
```

- **A `while` loop repeats its block for as long as its test is `True`.** `while True` never stops by itself, so the block must leave with `break`, which ends the loop at once and goes on after it.
- **`continue` ends the current pass and goes back to the loop's first line for the next one.** A loop uses it to pass over items it does not want, as `filter.py` passes over `70000`.

Tuples: Fixed Records, Unpacked by Position



kali@kali:~/python-essentials$

```
>>> addr = ("10.0.0.7", 443)
>>> addr[0]
'10.0.0.7'
>>> host, port = addr          # unpack
>>> host
'10.0.0.7'
>>> port
443
```

~/python-essentials/pairs.py

```
1 conns = [("10.0.0.7", 443), ("10.0.0.9", 22)]
2 for host, port in conns:  # one tuple per pass
3     print(host, "port", port)
```

kali@kali:~/python-essentials$

```
$ python3 pairs.py
10.0.0.7 port 443
10.0.0.9 port 22
```

- **A tuple, type `tuple`, is a fixed sequence of values between parentheses: once made, it cannot change.** It suits a record whose positions have meanings, such as an address and a port, and it is indexed like a list.
- **Unpacking assigns a sequence's items to names by position**, one name per item. A `for` line can unpack too: in `pairs.py`, each pass takes one tuple and splits it into `host` and `port`.
- **A network address is usually handled as a tuple of host and port**, so `("10.0.0.7", 443)` is the shape a program uses whenever it names one end of a connection.

Dictionaries: Looking Up a Value by Its Key

```
>>> SERVICES = {22: "ssh", 80: "http", 443: "https"}
```

key	value
22	'ssh'
80	'http'
443	'https'

1. get the value: `[]`

`SERVICES[443]` → `'https'`

2. test for the key: `in`

`8080 in SERVICES` → `False`

no key 8080: nothing to find

3. get with a default: `.get()`

`SERVICES.get(8080, "unknown")` → `'unknown'`

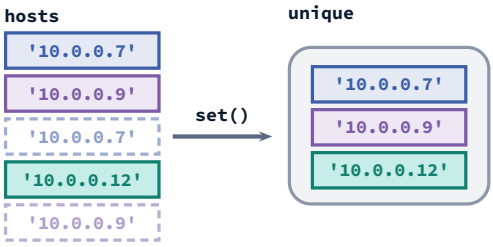
```
>>> SERVICES[8080] = "http-alt"
```

key	value
22	'ssh'
80	'http'
443	'https'
8080	'http-alt'

a new key: added at the end

- **A dictionary, type `dict`, maps each key to a value, written `{key: value}` between braces.** `SERVICES[443]` looks the key up, assigning to a new key adds a pair, and assigning to an existing key replaces its value.
- **`in` tests the keys, and `.get(key, default)` returns the default when the key is missing.** A lookup table like this turns a number into a name. `for port, name in SERVICES.items()` visits every pair, in the order they were added.

Sets: Each Member Exactly Once




```
kali@kali:~/python-essentials$
>>> hosts = ["10.0.0.7", "10.0.0.9", "10.0.0.7",
...          "10.0.0.12", "10.0.0.9"]
>>> unique = set(hosts)
>>> unique
{'10.0.0.7', '10.0.0.9', '10.0.0.12'}
>>> len(unique)
3
>>> "10.0.0.9" in unique
True
>>> unique.add("10.0.0.20")
>>> len(unique)
4
```

- **A set, type `set`, is a collection with no order and no repeats: each value is a member at most once.** `set(hosts)` keeps one of each, `in` asks whether a value is a member, and `add` puts one in.
- **A set prints in no fixed order**, so the order on your screen may differ from this one, and from one run to the next. An empty set is written `set()`, because `{}` makes an empty dictionary.

Common Mistakes: Colons and Indentation

what was typed	what Python prints

`port = 443`
`if port == 443`
`    print("https")`
	`File "/home/kali/python-essentials/colon.py", line 2`
	`  if port == 443`
	`                ^`
	`SyntaxError: expected ':'`

`port = 443`
`if port == 443:`
`|print("https")`
	`File "/home/kali/python-essentials/indent.py", line 3`
	`  print("https")`
	`  ^^^^^`
	`IndentationError: expected an indented block after 'if' statement on line 2`

`port = 443`
`if port = 443:`
`    print("https")`
	`File "/home/kali/python-essentials/assign.py", line 2`
	`  if port = 443:`
	`     ^^^^^^^^^^`
	`SyntaxError: invalid syntax. Maybe you meant '==' or ':=' instead of '='?`

- **These mistakes stop the file before its first line runs**, so nothing is printed but the error. Python reports the line and points at the spot with carets.
- Every `if`, `elif`, `else`, `for` and `while` line ends with a colon, and the line after it is indented. A test compares with `==`. A single `=` assigns, and an `if` cannot assign.

Common Mistakes: Indexes, Keys, and Ranges

what was typed	what Python prints

`ports = [22, 80, 443]`
`print(ports[0])`
`print(ports[3])`
	`22 # line 2 ran`
	[4 lines: the traceback, pointing at line 3]
	`IndexError: list index out of range`

`SERVICES = {22: "ssh", 80: "http", 443: "https"}`
`print(SERVICES[443])`
`print(SERVICES[8080])`
	`https # line 2 ran`
	[4 lines: the traceback, pointing at line 3]
	`KeyError: 8080`

`for i in range(1, 4):`
`  print(f"10.0.0.{i}")`
	`10.0.0.1`
	`10.0.0.2`
	`10.0.0.3 # no error, no 10.0.0.4`

- **These mistakes stop the script partway, after the lines before them have run**: an index past the end of a list raises `IndexError`, and a missing key raises `KeyError`. `.get()` never raises it.
- **The worst mistake prints no error at all.** `range(1, 4)` stops before 4, so the loop makes three addresses, not four. Count what a loop should produce, and check the output against it.

A Complete Script: Summarizing Connections

~/python-essentials/summary.py

```
SERVICES = {22: "ssh", 80: "http", 443: "https"}
lines = ["10.0.0.7:443", "10.0.0.9:22", "10.0.0.7:80",
         "10.0.0.12:8080", "10.0.0.9:70000"]
hosts = set()                  # an empty set
for line in lines:             # iteration: once per line
    host, text = line.split(":")
    port = int(text)
    if port < 0:               # selection: one branch runs
        print("Skipped", line, "(negative port)")
    elif port <= 65535:
        print(host, port, SERVICES.get(port, "unknown"))
        hosts.add(host)
    else:
        print("Skipped", line, "(port too large)")
print(len(hosts), "hosts")  # a count: no order
```

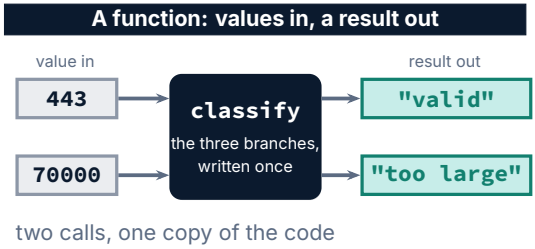
kali@kali:~/python-essentials$

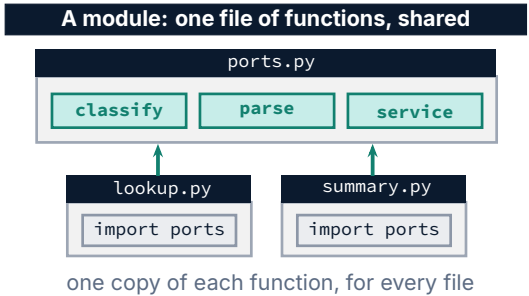
```
$ python3 summary.py
10.0.0.7 443 https
10.0.0.9 22 ssh
10.0.0.7 80 http
10.0.0.12 8080 unknown
Skipped 10.0.0.9:70000 (port too large)
3 hosts
```

- **One script uses every piece**: a dictionary names the services, a list holds the lines, a set gathers the hosts, a `for` loop repeats the work for each line, and one `if`, `elif`, `else` statement runs exactly one branch per line.
- **The last line prints how many hosts there are, not the set itself**, because a set has no fixed order. Counting keeps the output the same on every run, and the skipped line is left out of the count.

Functions and Modules

Functions and Modules: Naming Code for Reuse





- **A function is a named block of code that runs only when it is called.** A call hands it a value and gets one result back, so `classify` turns 443 into `"valid"` and 70000 into `"too large"`.
- **Written once, a function replaces every copy of its code**, so a fix is made in one place. Until now, `classify.py` and `summary.py` each held their own copy of the same three branches.
- **A module is a Python file whose functions other files use by importing it**, so `ports.py` shares one copy of `classify`, `parse` and `service` with every script.

Defining and Calling a Function

step	line	what happens
1	1	`def` creates `report`, and none of its body runs
2	5	call 1 binds `host='10.0.0.7'` and `port=443`
3	2	the body prints `10.0.0.7 port 443`
4	6	back after call 1, call 2 binds `host='10.0.0.9'` and `port=22`
5	2	the body prints `10.0.0.9 port 22`
6	7	back after call 2: prints `done`

~/python-essentials/report.py

```
1  def report(host, port):          # a definition
2      print(host, "port", port)    # the body
3
4
5  report("10.0.0.7", 443)          # call 1
6  report("10.0.0.9", 22)           # call 2
7  print("done")
```

kali@kali:~/python-essentials$

```
$ python3 report.py
10.0.0.7 port 443
10.0.0.9 port 22
done
```

- **`def` defines a function: its name, its parameters in parentheses, a colon, then an indented body.** A **parameter** is a name the body uses for a value it will be given. The `def` line runs none of the body.
- **A call runs the body: the function's name, then one argument, a value, for each parameter.** The run jumps into the body, then comes back to the line after the call, so one body prints twice.

Return Values: Handing a Result Back

~/python-essentials/labels.py

```
1  def classify(port):
2      if port < 0:
3          return "negative"
4      elif port <= 65535:
5          return "valid"
6      else:
7          return "too large"
8
9
10 label = classify(443)  # gets "valid"
11 print("443 is", label)
12 print("70000 is", classify(70000))
```

step	line	what happens in `classify(443)`
1	10	the call starts: `port = 443`
2	2	`port < 0` is `False`
3	4	`port <= 65535` is `True`
4	5	returns `"valid"` to line 10: `label = "valid"`
5	11	prints `443 is valid`

kali@kali:~/python-essentials$

```
$ python3 labels.py
443 is valid
70000 is too large      # the second call
```

- **`return` ends the call and hands a value back to the line that made it**, where the value takes the call's place: `label = classify(443)` stores `"valid"`. A function that reaches no `return` hands back `None`.
- **`print` shows a value to a person, and `return` gives it to the program**, which can store it, test it or pass it on. A function can also return two values as one tuple, which the caller unpacks by position.

Keyword Arguments and Default Values

~/python-essentials/address.py

```
1  def address(n, prefix="10.0.0."):
2      return f"{prefix}{n}"
3
4
5  print(address(7))
6  print(address(7, "192.168.1."))
7  print(address(n=7, prefix="172.16.0."))
8  print(address(prefix="172.16.0.", n=7))
```

n	prefix
7	"10.0.0."
7	"192.168.1."
7	"172.16.0."
7	"172.16.0."

kali@kali:~/python-essentials$

```
$ python3 address.py
10.0.0.7
192.168.1.7
172.16.0.7
172.16.0.7
```

by position: the first argument fills the first parameter
by name: `n=7` fills `n`, wherever it is written

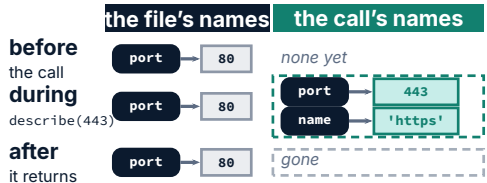
from the default in line 1

- **A positional argument fills the parameter in its place, and a keyword argument names the parameter it fills**, as in `n=7`, so its order no longer matters. Keyword arguments follow positional ones, and `print(..., sep=":")` already used one.
- **A default value makes a parameter optional**: a call that leaves `prefix` out gets `"10.0.0."`, so `address(7)` gives `10.0.0.7`.
- **A function's name without parentheses is the function itself, not a call**, a value like any other. Later tools hand functions to other functions this way, often as keyword arguments.

Scope: Where a Name Exists

~/python-essentials/scope.py

```
1  SERVICES = {22: "ssh", 80: "http", 443: "https"}
2  port = 80
3
4
5  def describe(port):
6      name = SERVICES.get(port, "unknown")
7      return f"{port}/{name}"
8
9
10 print(describe(443))
11 print(port)
```



- **A name's scope is the part of the program where it can be used.** A name assigned inside a function, its parameters included, is a **local variable**: it exists only while one call runs.
- **A function can read the names defined at the top of its file**, which is how `describe` uses `SERVICES`. A fixed value kept there is a constant, and its name is written in capitals.
- **Assigning inside a function never changes a name outside it.** The call's `port` is 443 while the file's `port` stays 80, as the run's last line shows.

Modules: Importing a File's Functions

~/python-essentials/ports.py

```
1  """ports.py: classify ports and name their services."""
2  SERVICES = {22: "ssh", 80: "http", 443: "https"}
3
4
5  def classify(port):
       [lines 6–11: the three branches, as in labels.py]
12
13
14 def parse(line):
15     host, text = line.split(":")
16     return host, int(text)
17
18
19 def service(port):
20     return SERVICES.get(port, "unknown")
```

~/python-essentials/lookup.py

```
1  import ports  # runs ports.py once
2
3  print(ports.classify(443))
4  print(ports.parse("10.0.0.7:443"))
5  print(ports.service(22))
```

kali@kali:~/python-essentials$

```
$ python3 lookup.py
valid
('10.0.0.7', 443)  # a tuple
ssh
```



- `import ports` **runs the file** `ports.py` **once and binds the name** `ports` **to the module it made.** Everything the file defined is then reached through that one name.
- **A dot reaches a name inside a module**: `ports.classify(443)` calls the module's own function. `ports.parse` returns a tuple, `('10.0.0.7', 443)`, which a caller can unpack into two names.

The Module Search Path: `sys.path`

~/python-essentials/where.py

```python
import sys   # standard library

for folder in sys.path:
    print(folder)
```

kali@kali:~/python-essentials$

```
(.venv) $ python3 where.py
/home/kali/python-essentials                                  # 1: the script's own folder
/usr/lib/python313.zip
/usr/lib/python3.13                                           # 2: the standard library
/usr/lib/python3.13/lib-dynload
/home/kali/python-essentials/.venv/lib/python3.13/site-packages  # 3: packages pip installed
```

- **The standard library is the set of modules that come with Python**, imported the same way. `import sys` gives `sys.path`, the list of folders Python searches, in order, for a module to import.
- **The script's own folder is searched first**, then the standard library, then the packages `pip` installed. So a file of yours named like a standard module, such as `socket.py`, hides the real one.

A Script Built From Functions: The Main Guard

~/python-essentials/summary.py

```
1  """summary.py: summarize connection lines."""
2  import ports                  # runs ports.py once
3
4  LINES = ["10.0.0.7:443", "10.0.0.9:22", "10.0.0.7:80"]
5
6
7  def main():                   # the script's work
8      for line in LINES:
9          host, port = ports.parse(line)
10         print(host, port, ports.service(port))
11
12
13 if __name__ == "__main__":   # True only when run
14     main()
```

kali@kali:~/python-essentials$

```
$ python3 summary.py
10.0.0.7 443 https # main() ran
10.0.0.9 22 ssh
10.0.0.7 80 http
$ python3 -c 'import summary'
```

the import printed nothing: `main()` did not run

~/python-essentials/noguard.py

```
[lines 1–12: the same as summary.py]
13 main()    # no guard
```

kali@kali:~/python-essentials$

```
$ python3 -c 'import noguard'
[the run's 3 lines again: main() ran]
```

- **Python sets `__name__` to `"__main__"` in the file it runs, and to the module's name in a file it imports.** So `main()` runs when `summary.py` is run, not when it is imported. `noguard.py` calls `main()` with no guard, so importing it runs `main()` too.
- **A script keeps one order: a docstring, imports, constants, functions, then the guard.** A **docstring** is a string on the first line of a file or function that says what it is for.

Common Mistakes: Calls and Returns

what was typed	what Python prints

```python
def parse(line):
    host, text = line.split(":")
    return host, int(text)


host, port = parse()
```

```
[4 lines: the traceback, pointing at line 6]
TypeError: parse() missing 1 required positional argument: 'line'
```

```python
def address(n):
    print(f"10.0.0.{n}")


host = address(7)
print("host is", host)
```

```
10.0.0.7          # printed inside address
host is None      # no error, and no value
```

- **A call must give a value to every parameter that has no default.** `parse()` gives `line` nothing, so the call stops the script with a `TypeError` that names the missing parameter.
- **A function that prints its result instead of returning it hands back `None`, and no error says so.** The address reaches the screen, but `host` holds `None`. Return any value the program still needs.

Common Mistakes: Uncalled Functions and Local Names

what was typed

```
1 def address(n):
2     return f"10.0.0.{n}"
3
4
5 print(address)
```

what Python prints

```
<function address at ...>  # changes every run
```

what was typed

```
1 def parse(line):
2     host, text = line.split(":")
3     return host, int(text)
4
5
6 parse("10.0.0.7:443")
7 print(host)
```

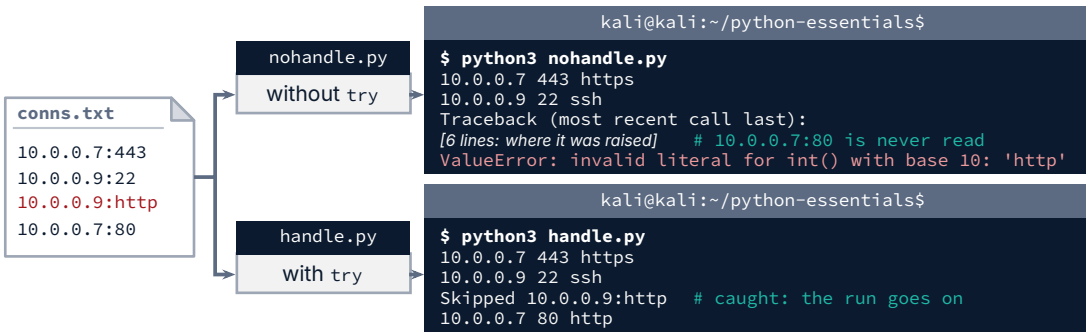
what Python prints

```
[4 lines: the traceback, pointing at line 7]
NameError: name 'host' is not defined
```

- **A function's name without parentheses is the function, not a call**, so `print(address)` describes the function and raises nothing. The number after `at` is where it sits in memory, and it changes from run to run.
- **A local variable is gone once its call returns.** `host` existed only inside `parse`, so the last line stops with a `NameError`. Keep a result by assigning what the call returns: `host, port = parse(...)`.

Files and Exceptions

Files and Exceptions: Real Input, Recoverable Errors



- **A file is named data kept on disk, outside any program**, so it is still there when a run ends. Until now the connection lines were typed into the script, and new input meant editing the code.
- **An exception is the error a failing line raises.** Unhandled, it stops the script with a traceback, so one bad line, `10.0.0.9:http`, ends the whole run.
- **A handled exception is caught, and the run goes on**: the bad line is skipped and the next one is read.

Opening and Reading a File

conns.txt

```
pass 1  10.0.0.7:443\n
pass 2  10.0.0.9:22\n
pass 3  10.0.0.9:http\n
pass 4  10.0.0.7:80\n
```

kali@kali:~/python-essentials$

```
>>> with open("conns.txt") as f:
...     text = f.read()
...
>>> f.closed
True   # closed after the block
```

~/python-essentials/echo.py

```
1 with open("conns.txt") as f:
2     for line in f:
3         print(line)
```

kali@kali:~/python-essentials$

```
$ python3 echo.py
10.0.0.7:443
# the line's own \n
10.0.0.9:22

10.0.0.9:http

10.0.0.7:80
```

~/python-essentials/read.py

```
1 with open("conns.txt") as f:
2     for line in f:
3         print(line.strip())
```

kali@kali:~/python-essentials$

```
$ python3 read.py
10.0.0.7:443
10.0.0.9:22
10.0.0.9:http
10.0.0.7:80  # no blank rows
```

- **`open` connects the program to a file and returns a file object**, which `with ... as f` names `f`. When the block ends, `with` closes the file, even if an exception was raised inside it.
- **`for line in f` reads one line per pass, and each line keeps its `\n`**, so `print` shows a blank row after it. `strip()` removes it. `f.read()` returns the whole file as one string.
- **A file name with no folder is looked up in the folder the script is run from**, not in the script's own folder.

Writing a File: Modes `w` and `a`

~/python-essentials/write.py

```
1 with open("report.txt", "w") as f:
2     f.write("10.0.0.7 443 https\n")
```

```
kali@kali:~/python-essentials$
$ python3 write.py
$ python3 write.py
$ cat report.txt
10.0.0.7 443 https
```

report.txt

```
10.0.0.7 443 https
```

after two runs: one line, the first erased and written again (teal)

~/python-essentials/append.py

```
1 with open("report.txt", "a") as f:
2     f.write("10.0.0.7 443 https\n")
```

```
kali@kali:~/python-essentials$
$ rm report.txt
$ python3 append.py
$ python3 append.py
$ cat report.txt
10.0.0.7 443 https
10.0.0.7 443 https
```

report.txt

```
10.0.0.7 443 https
10.0.0.7 443 https
```

after two runs: the first run's line kept, the second's added (teal)

r read: the default	**w** write: empties the file first	**a** append: adds to the end	**b** binary: added to r, w or a

- **The mode, the second argument to `open`, says what the program will do with the file.** Without one, `open` reads.
- **`"w"` empties the file before the first write**, and creates it if it is missing, so every run of `write.py` leaves one line. `"a"` keeps what is there and adds to the end.
- **`write` writes exactly the string it is given**, with no newline of its own, so the `\n` is written by hand.

Handling an Exception: `try` and `except`

step	line	what happens to the bad line
1	4	`line` is `"10.0.0.9:http\n"`
2	5	`try` starts the block
3	6	`parse` raises a `ValueError`
4	8	`except ValueError` matches it
5	9	prints `Skipped 10.0.0.9:http`
6	4	the loop takes the next line

line 7 never runs: the raise skipped the rest of the block

kali@kali:~/python-essentials$

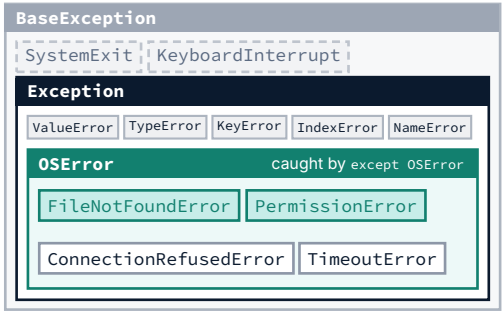
```
$ python3 handle.py
10.0.0.7 443 https
10.0.0.9 22 ssh
Skipped 10.0.0.9:http  # step 5
10.0.0.7 80 http       # step 6, then on
```

~/python-essentials/handle.py

```
1  import ports
2
3  with open("conns.txt") as f:
4      for line in f:
5          try:   # lines 6 and 7 may raise
6              host, port = ports.parse(line.strip())
7              print(host, port, ports.service(port))
8          except ValueError:   # on a ValueError
9              print("Skipped", line.strip())
```

- **`try` runs its block, and a line that raises skips the rest of the block**: the run jumps to the `except` that names the exception's class. `int("http")` raises a `ValueError`, so line 7 is skipped on that pass.
- **After the `except` block, the run continues below the whole statement**, so the loop reads the next line. When nothing is raised, the `except` block is skipped.

The Exception Hierarchy: Catching a Family of Errors



~/python-essentials/family.py

```
1 for name in ["missing.txt", "locked.txt"]:
2     try:
3         with open(name) as f:
4             print(f.read())
5     except OSError as err:   # err: which one
6         print(err)
```

kali@kali:~/python-essentials$

```
$ printf 'secret\n' > locked.txt
$ chmod 000 locked.txt     # no one may read it
$ python3 family.py
[Errno 2] No such file or directory: 'missing.txt'
[Errno 13] Permission denied: 'locked.txt'
```

- **Exceptions are classes in a hierarchy, and each class is a kind of every class around it**: a `FileNotFoundError` is an `OSError`, and an `Exception`.
- **`except` catches the class it names and every class inside it**, so one `except OSError` catches both failures. `as err` names the exception, and `print(err)` shows its message.
- **A bare `except:` catches everything**, even Ctrl-C's `KeyboardInterrupt`, so always name the class.

Raising an Exception, and Exiting With a Message

~/python-essentials/raise.py

```
1  def check(port):
2      if not 0 <= port <= 65535:
3          raise ValueError(f"out of range: {port}")
4      return port  # skipped when line 3 raises
5
6
7  try:              # the caller
8      print("Using port", check(70000))
9  except ValueError as err:
10     print("Skipped:", err)
```

kali@kali:~/python-essentials$

```
$ python3 raise.py
Skipped: out of range: 70000   # line 3's message
```

~/python-essentials/stop.py

```
1  port = 70000
2  if not 0 <= port <= 65535:
3      raise SystemExit(f"out of range: {port}")
4  print("Using port", port)   # never runs
```

kali@kali:~/python-essentials$

```
$ python3 stop.py
out of range: 70000  # one line, no traceback
$ echo $?            # the exit status
1
```

- `raise` **creates an exception and raises it, just as a failing built-in does.** `check` raises a `ValueError` with its own message, and the caller's `except` catches it like any other.
- `raise SystemExit("...")` **ends the script with that message, no traceback, and exit status 1**, which `echo $?` shows. An exception left unhandled also ends the script, but with a traceback.

A Complete Script: Summarizing a Connections File

~/python-essentials/summary.py

```
    [lines 1–4: the docstring, then import ports]
5   def main():
6       try:          # the file may be missing
7           with open("conns.txt") as f:
8               lines = f.read().splitlines()
9       except FileNotFoundError:
10          raise SystemExit("conns.txt not found")
11      for line in lines:
12          try:      # the line may be bad
13              host, port = ports.parse(line)
14              print(host, port, ports.service(port))
15          except ValueError:
16              print("Skipped", line)
    [lines 17–20: two blank lines, then the main guard]
```

kali@kali:~/python-essentials$

```
$ python3 summary.py
10.0.0.7 443 https
10.0.0.9 22 ssh
Skipped 10.0.0.9:http  # line 16
10.0.0.7 80 http
$ mv conns.txt saved.txt # now missing
$ python3 summary.py
conns.txt not found    # line 10
$ echo $?
1
```

- **Each step that can fail sits in its own `try`**: opening the file, which can be missing, and parsing a line, which can be bad. A bad line is skipped, and a missing file ends the run with one clear message.
- **`splitlines()` splits the file's text at each `\n` and drops it**, so every line arrives without its newline and needs no `strip()`.

Common Mistakes: Files and Exceptions

what was typed	what the run prints

```
with open("conns.txt", "w") as f:
    for line in f:
        print(line.strip())
```

```
[4 lines: the traceback, pointing at line 2]
io.UnsupportedOperation: not readable
$ wc -c conns.txt
0 conns.txt           # emptied before the error
$ python3 read.py     # prints nothing: no lines left
```

```
text = "443"
try:
    port = int(text)
    print("Using port", prt)  # typo
except Exception:
    print("Not a port:", text)
```

```
$ python3 broad.py
Not a port: 443       # wrong: a NameError was hidden
```

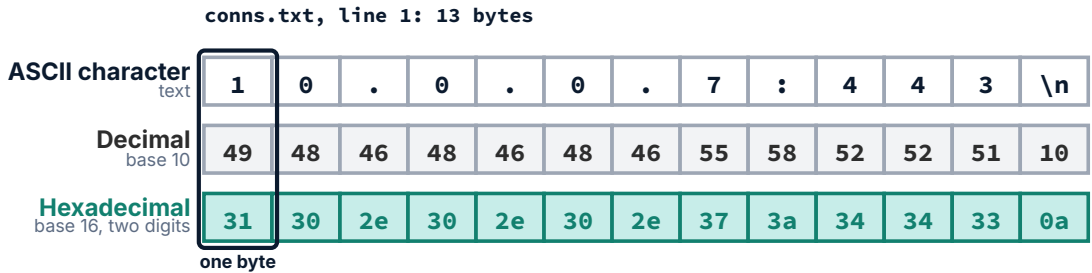
```
[lines 1–4: the same as above]
except ValueError:
    print("Not a port:", text)
```

```
$ python3 narrow.py   # right: the typo is named
[4 lines: the traceback, pointing at line 4]
NameError: name 'prt' is not defined. Did you mean: 'port'?
```

- **Opening a file with `"w"` empties it at once**, before any line reads it, so the error that follows arrives after the data is gone. Open a file you only read with no mode.
- **`except Exception` also catches mistakes in your own code**: the misspelled `prt` raises a `NameError`, and the script reports 443 as not a port. Name the exception you expect, and the typo is reported with its line.

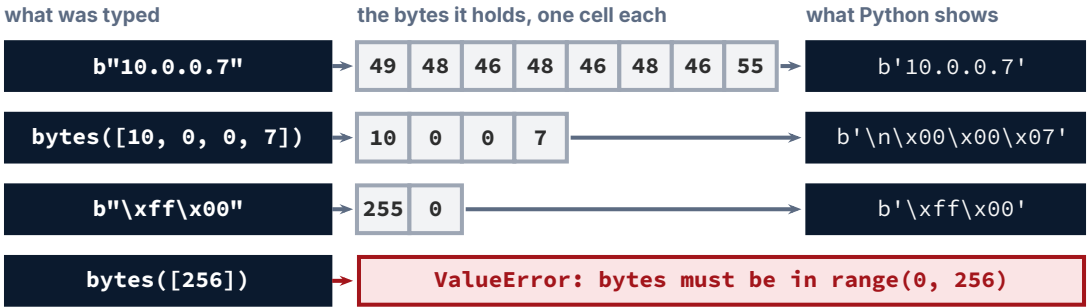
Bytes and Binary Data

Bytes and Binary Data: What a File Really Holds



- A **byte** is the unit every file is stored in: a number from 0 to 255. Whatever program wrote a file, the disk holds a sequence of bytes and nothing else.
- **Text is one way of reading bytes.** ASCII, the table that numbers English letters, digits and punctuation from 0 to 127, says byte 49 is the character `1`. In **hexadecimal**, the same byte is `31`: every byte takes exactly two digits, from `00` to `ff`.
- **Binary data is bytes meant as numbers rather than characters**, like an address stored as four numbers from 0 to 255. Python holds both kinds in one type, `bytes`.

The `bytes` Type: A Sequence of Numbers From 0 to 255



- A `bytes` object is **a sequence of numbers from 0 to 255 that cannot change**. A literal puts a `b` before the quotes, and `bytes()` builds one from a list of numbers.
- **A literal holds ASCII characters only.** Any other byte is written as `\x` and two hex digits, so `\xff` is 255. A number above 255 raises a `ValueError`.
- **Python shows a byte as its character when that character is printable, and as an escape otherwise**, so 10, the newline, appears as `\n`. `len` counts bytes, not characters.

Indexing Gives an `int`, Slicing Gives `bytes`

```
data = b"192.168.1.10"
```




```
data[0]   49
data[0:3] b'192'
```

```
kali@kali:~/python-essentials$
[addr: the four bytes 10, 0, 0, 7]
>>> a, b, c, d = addr
>>> a, d
(10, 7) # ints
>>> for n in b"192":
...     print(n)
...
49
57
50  # one int per pass
```

- **Indexing gives an int, the byte's value, and slicing gives a shorter bytes object.** So `data[0]` is 49, where the same index on a string would give a string.
- **Looping over bytes, or unpacking them, hands out ints**: four bytes unpack into four numbers. `+`, `in` and `len` work as they do on a string.
- A `bytes` object cannot change, so `addr[3] = 9` raises a `TypeError`. A `bytearray` is **the changeable form**: `bytearray(addr)` copies it, item assignment works, and `bytes()` turns it back.

Displaying Bytes as Hex: `.hex()` and `bytes.fromhex`




```
kali@kali:~/python-essentials$
[addr: the four bytes 10, 0, 0, 7]
>>> addr.hex()
'0a000007'
>>> addr.hex(" ")
'0a 00 00 07' # one byte per pair
>>> bytes.fromhex("0a 00 00 07")
b'\n\x00\x00\x07'
>>> bytes.fromhex("0a 00 00 07") == addr
True  # the same bytes
>>> f"{addr[3]:02x}"
'07'  # one byte, two digits
```

- **Python prints bytes as a bytes literal**: printable bytes as characters, the rest as escapes, so `bytes([10, 0, 0, 7])` prints as `b'\n\x00\x00\x07'`. Hex shows every byte the same way.
- `.hex()` **returns the bytes as hex digits**, two per byte, and `.hex(" ")` puts a space between bytes. `bytes.fromhex` reads such a string back and ignores the spaces, so the round trip returns the same bytes.
- One byte is an int, and the f-string spec `02x` formats it as two hex digits.

Binary Files: Modes `rb` and `wb`

~/python-essentials/write.py

```
1 with open("hosts.bin", "wb") as f:
2     f.write(bytes([10, 0, 0, 7]))
3     f.write(bytes([10, 0, 0, 9]))
```

kali@kali:~/python-essentials$

```
$ python3 write.py
$ wc -c hosts.bin
8 hosts.bin  # two writes of four bytes
```

rb read bytes: `.read()` returns the whole file as `bytes`

~/python-essentials/read.py

```
1 with open("hosts.bin", "rb") as f:
2     data = f.read()
3 print(data)
4 print(len(data), data.hex(" "))
```

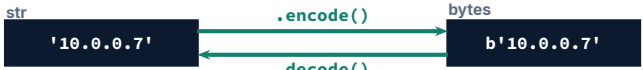
kali@kali:~/python-essentials$

```
$ python3 read.py
b'\n\x00\x00\x07\n\x00\x00\t'   # line 3
8 0a 00 00 07 0a 00 00 09       # line 4
```

wb write bytes: empties the file first, then takes `bytes` only

- **Adding b to the mode opens a file in binary mode**: `"rb"` reads and `"wb"` writes, and the data passes as `bytes`, unchanged in either direction.
- In binary mode `write` takes bytes and writes exactly those bytes, so two four-byte writes make the 8-byte file that `wc -c` counts.
- Like `"w"`, `"wb"` **empties an existing file before the first write**. `.read()` with no argument returns the whole file as one `bytes` object.

Text and Bytes: `encode` and `decode`



kali@kali:~/python-essentials$

```
>>> "10.0.0.7".encode()
b'10.0.0.7'
>>> b"10.0.0.7".decode()
'10.0.0.7'
>>> "10" + b"10"    # never converted for you
Traceback (most recent call last):
  [3 lines: where it happened, and the line itself]
TypeError: can only concatenate str (not "bytes") to str
```

~/python-essentials/decode.py

```
1 with open("conns.txt", "rb") as f:
2     raw = f.read()
3 print(raw[:13])
4 text = raw.decode()
5 with open("conns.txt") as f:
6     print(text == f.read())
```

kali@kali:~/python-essentials$

```
$ python3 decode.py
b'10.0.0.7:443\n'   # line 3: bytes
True # the text "r" gives
```

- `encode` **turns text into bytes**, and `decode` **turns bytes back into text**, both with UTF-8 unless told otherwise. Python never converts between the two on its own, so `"10" + b"10"` raises a `TypeError`.
- **Text mode is binary mode plus a decode**: `conns.txt` read with `"rb"` and decoded is exactly the string that `"r"` returns.
- Bytes that are not text cannot be decoded: `bytes([192, 168, 1, 10]).decode()` raises a `UnicodeDecodeError`. `errors="replace"` puts a marker in place of each bad byte instead.

A Complete Script: Reading an Address File

~/python-essentials/hosts.py

```
    [lines 1–3: the docstring, then two blank lines]
4   def main():
5       try:
6           with open("hosts.bin", "rb") as f:
7               data = f.read()
8       except FileNotFoundError:
9           raise SystemExit("hosts.bin not found")
10      if len(data) % 4 != 0:    # a partial address
11          raise SystemExit("hosts.bin ends mid-address")
12      for i in range(0, len(data), 4):      # step 4
13          chunk = data[i:i + 4]
14          a, b, c, d = chunk    # four ints
15          print(f"{a}.{b}.{c}.{d}", chunk.hex(" "))
    [lines 16–19: two blank lines, then the main guard]
```

kali@kali:~/python-essentials$

```
$ python3 hosts.py
10.0.0.7 0a 00 00 07   # line 15
10.0.0.9 0a 00 00 09
$ cp hosts.bin full.bin
$ head -c 7 full.bin > hosts.bin
$ python3 hosts.py
hosts.bin ends mid-address # line 11
$ echo $?
1
```

- **The script checks its input before it uses it**: a missing file, or a length that is not a multiple of 4, ends the run with one message before any address is printed.
- `range(0, len(data), 4)` **steps through the file four bytes at a time**, so each slice holds one address and unpacks into four ints. `head -c 7` copies only the first 7 bytes, which cuts the second address short.

Common Mistakes: Mixing `str` and `bytes`

what was typed	what the run prints

`with open("hosts.bin", "wb") as f:`
`    f.write("10.0.0.7")`
	`$ python3 wbstr.py`
	[4 lines: the traceback, pointing at line 2]
	`TypeError: a bytes-like object is required, not 'str'`
	`$ wc -c hosts.bin`
	`0 hosts.bin    # emptied before the error`

`with open("conns.txt", "rb") as f:`
`    data = f.read()`
`if data[0] == b"1":`
`    print("starts with 1")`
`else:`
`    print("no match")`
	`$ python3 first.py`
	`no match       # wrong: data[0] is 49, an int`

[lines 1–2: the same as above] `if data[0:1] == b"1":` *[lines 4–6: the same as above]*	`$ python3 sliced.py` `starts with 1  # right: a slice is bytes`

- A file opened with `"wb"` **takes bytes only**: writing a string raises a `TypeError`, and by then the file is already empty. Adding bytes to a string, as in `frame = ""` then `frame += b"\x0a"`, fails the same way.
- **An index gives an int, and an int never equals a bytes object**, so `data[0] == b"1"` is `False` and nothing is raised. Compare a slice, `data[0:1]`, or the number 49.

Command-Line Tools

Command-Line Tools: Arguments From the Shell

bash splits the command line at each space



~/python-essentials/show.py

```
1 import sys
2
3 print(sys.argv)
```

kali@kali:~/python-essentials$

```
$ python3 show.py conns.txt --host 10.0.0.7
['show.py', 'conns.txt', '--host', '10.0.0.7']    # a list of str
$ python3 show.py
['show.py']                                       # the name alone
```

- A **command-line argument** is a word typed after the script's name: bash splits the command line at each space and hands every word after `python3` to the script, as text.
- Part 5's `summary.py` names `conns.txt` in its code, so reading another file means editing the script. **A command-line tool takes its input as arguments, so one script runs on any file.**
- Python puts the words in a list, `sys.argv`. `argparse` **reads that list for you and checks it**, and `ipaddress` checks that a word is a real address.

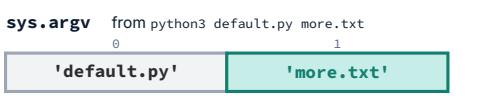
`sys.argv`: The Command Line as a List of Strings

~/python-essentials/default.py

```
1 import sys
2
3 path = sys.argv[1] if len(sys.argv) > 1 else "conns.txt"
4 print("Reading", path)
```

kali@kali:~/python-essentials$

```
$ python3 default.py more.txt
Reading more.txt
$ python3 default.py
Reading conns.txt
```



`len(sys.argv) > 1` is True, so `path = 'more.txt'`



`len(sys.argv) > 1` is False, so `path = 'conns.txt'`

- `sys.argv` is **the list of words the script received**, its own name at index 0, so the first argument is `sys.argv[1]`. Every item is a `str`, even `443`.
- A **conditional expression**, `A if test else B`, **chooses between two values**: A when the test is true, B otherwise. Line 3 falls back to `conns.txt` when no file is given.
- When an argument is required, test `len(sys.argv) < 2` and `raise SystemExit("usage: ...")`, so the run ends on one usage line instead of an `IndexError` from `sys.argv[1]`.

`argparse`: Declaring What a Tool Accepts

~/python-essentials/declare.py

```
1  import argparse
2
3  parser = argparse.ArgumentParser(
4      description="Show a file name.")                (1)
5  parser.add_argument("file", help="file to read")    (2)
6  args = parser.parse_args()
7  print(args)
8  print("Reading", args.file)
```

kali@kali:~/python-essentials$

```
$ python3 declare.py conns.txt
Namespace(file='conns.txt')  # line 7
Reading conns.txt  # line 8: args.file
$ python3 declare.py -h
usage: declare.py [-h] file

Show a file name. (1)

positional arguments:
  file        file to read (2)

options:  # -h is added by argparse
  -h, --help  show this help message and exit
```

- `argparse` **is the standard-library module that reads the command line from a declaration**: `ArgumentParser` makes the parser, and each `add_argument` declares one argument by name.
- `parse_args()` **reads the command line and returns an object with one attribute per argument**, so the file name is `args.file`. Python prints that object as `Namespace(...)`.
- **The help is generated from the declaration**: `-h` prints the usage line, the description (1) and each argument's `help=` text (2). argparse gives every parser `-h` itself.

Options, Types, and the Usage Error

~/python-essentials/options.py

```
    [lines 1–3: import argparse, then make the parser]
4   parser.add_argument("file", nargs="?",    # optional
5                       default="conns.txt")
6   parser.add_argument("--port", type=int, default=443,
7                       help="port to look for")
8   args = parser.parse_args()
9   print(args.file, args.port, type(args.port))
```

kali@kali:~/python-essentials$

```
$ python3 options.py
conns.txt 443 <class 'int'>  # defaults
$ python3 options.py more.txt --port 22
more.txt 22 <class 'int'>    # given
```

kali@kali:~/python-essentials$

```
$ python3 options.py --port http
usage: options.py [-h] [--port PORT] [file]
options.py: error: argument --port: invalid int value: 'http'
$ echo $?
2      # argparse: wrong usage
```

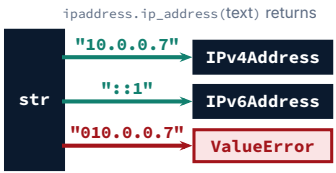
kali@kali:~/python-essentials$

```
$ python3 need.py
usage: python3 need.py FILE

$ echo $?
1  # raise SystemExit
```

- An argument whose name starts with `--` is an **option**: it is given by name, in any position, and may be left out, so `default=` supplies its value.
- `type=int` **hands argparse a function, uncalled**, and argparse calls `int` on each value it reads.
- **A value argparse cannot convert ends the run with the usage line, an error and exit status 2**, the status bash's own commands use for wrong usage. `raise SystemExit` exits with 1.

`ipaddress`: Checking an Address by Converting It



kali@kali:~/python-essentials$

```
[ipaddress imported]
>>> ipaddress.ip_address("10.0.0.7")
IPv4Address('10.0.0.7')
>>> ipaddress.ip_address("::1")
IPv6Address('::1')
>>> ipaddress.ip_address("010.0.0.7")
Traceback (most recent call last):
[5 lines: where it happened, inside ipaddress]
ValueError: '010.0.0.7' does not appear to be an IPv4 or IPv6 address
```

kali@kali:~/python-essentials$

```
$ python3 check.py 10.0.0.256     # check.py: add_argument("host", type=ipaddress.ip_address)
usage: check.py [-h] host
check.py: error: argument host: invalid ip_address value: '10.0.0.256'
```

- `ipaddress.ip_address` **converts text into an address object**, an `IPv4Address` or an `IPv6Address` (`.version` gives 4 or 6), and raises a `ValueError` when the text is not an address, a leading zero included.
- Passed as `type=ipaddress.ip_address`, it **makes argparse check the address**: a bad one ends the run with the usage error and exit status 2, before any of the script's own code runs.

Common Mistakes: Command-Line Arguments

what was typed	what the run prints

```
1 import sys
2
3 if sys.argv[1] == 443:
4     print("HTTPS")
5 else:
6     print("not 443")
```

```
$ python3 compare.py 443
not 443   # wrong: "443" is a str
```

```
[lines 1–2: the same as above]
3 if int(sys.argv[1]) == 443:
[lines 4–6: the same as above]
```

```
$ python3 fixed.py 443
HTTPS     # right: int() converts the text
```

```
1 import sys
2
3 with open(sys.argv[0]) as f:
4     print(f.read())
```

```
$ python3 zero.py conns.txt   # its own source
import sys

with open(sys.argv[0]) as f:
    print(f.read())
```

- **Every argument is text, and text never equals a number**, so `sys.argv[1] == 443` is `False` and nothing is raised. Convert first, or let argparse convert with `type=int`.
- `sys.argv[0]` **is the script's own name**, so `open(sys.argv[0])` reads the script itself, and nothing is raised. The first argument is `sys.argv[1]`.

A Complete Tool: Summarizing Any Connections File

~/python-essentials/summary.py

```
[lines 1–9: the docstring, three imports, def main(), the parser]
10      parser.add_argument("file", nargs="?",
11                          default="conns.txt")
12      parser.add_argument("--host",
13                          type=ipaddress.ip_address)
14      args = parser.parse_args()  # bad --host: exit 2
[lines 15–19: read the file, or exit if it is missing]
20      for line in lines:
21          try:
22              host, port = ports.parse(line)
23          except ValueError:
24              print("Skipped", line)
25              continue  # next line
26          if args.host is None or host == args.host:
27              print(host, port, ports.service(port))
[lines 28–31: two blank lines, then the main guard]
```

kali@kali:~/python-essentials$

```
$ python3 summary.py
10.0.0.7 443 https
10.0.0.9 22 ssh
Skipped 10.0.0.9:http
10.0.0.7 80 http
$ python3 summary.py more.txt
10.0.0.5 22 ssh
Skipped 10.0.0.300:80  # not an address
10.0.0.5 443 https
$ python3 summary.py --host 10.0.0.9
10.0.0.9 22 ssh   # from conns.txt
Skipped 10.0.0.9:http
$ python3 summary.py gone.txt
gone.txt not found
$ echo $?
1    # SystemExit, not argparse
```

- **The tool checks its input before any work**: `file` defaults to `conns.txt`, a bad `--host` ends the run with exit status 2, and `ports.parse` now converts each host in the file, so a bad one is skipped.
- argparse sets `args.host` to `None` when no `--host` is given. `args.host is None` **tests for exactly that value**, so line 26 prints every line unless a host was asked for.

Key Takeaways

- **Every project gets a private Python**, a virtual environment, so the packages it installs never reach Kali's own Python or another project.
- **Python never converts a value on its own**: a port typed in stays a `str`, and a file read with `"rb"` stays `bytes`, until you convert it.
- **A traceback is read from the bottom up**: its last line names the exception, and that class, never `Exception`, is the one to catch.
- **Write a step once, as a function, and import it where it is needed**, so a fix is made in one place instead of in every copy.
- **A tool checks its input by converting it, before any work**, so a bad port or address ends the run with one message, not a traceback.
- **The worst mistake prints no error at all**: `sys.argv[1] == 443` is `False`, and nothing is raised. Decide what the run should print, then check it.

How the Practical Guide Uses This

1. **The Practical Guide is what you practice from**, in the same two sittings: the language first, then the tool. Some exercises you write from scratch, and others start from a file with `# TODO` gaps for you to complete.
2. **The Submission worksheet is filled in as you go**: for each exercise, your code, what it printed, one screenshot of the run, and short written answers.
3. **The Home Task is done alone, at home**, and every task in it is required.

Two documents sit beside you while you work rather than being read through: the **Tooling Reference** covers every tool used here in full, including what today left out, such as list comprehensions and `choices=`, and the **Troubleshooting FAQ** is organized by the symptom in front of you.

Open the Practical Guide

`Module0_Session5_Practical_Guide.md`

Module 0 — Session 5
Python Essentials for Cybersecurity